NLA Note

1 Background

Floating Point ($\mathbb{F}$): A number = mantissa $\times$ base$^{\text{exponent}}$ (e.g. $2048 = 1 \times 2^{11}$ mantissa=1, base=2, exponent=11)

Assumptions: There is $\varepsilon_m > 0$ (machine epsilon) **A1**: $\forall \alpha \in \mathbb{R}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $fl : \mathbb{R} \to \mathbb{F}$ by $fl(\alpha) = \alpha(1+\varepsilon)$

A2: $\forall \alpha, \beta \in \mathbb{F}, \forall * \in \{+, -, \times, \div\}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $\alpha \circledast \beta = (\alpha * \beta)(1+\varepsilon)$

A1: The rounding error in α is relative to the size of α ($|\alpha - fl(\alpha)| < |\alpha|\varepsilon_m$) A2: The error in $\alpha \circledast \beta$ is relative to the size of $\alpha * \beta$ ($|\alpha \circledast \beta - \alpha * \beta| \le |\alpha * \beta|\varepsilon_m$)

IEEE 754: 64-bit: denote $\mathbb{F}$. 1-bit: sign 52-bit: mantissa 11-bit: exponent (base 2). $\varepsilon_m \approx 2.22 \times 10^{-16}$

Cost of Algorithm (C): $C :=$ number of floating point operations (FLOPs) If $C(n) \le \alpha f(n), \Rightarrow C(n) = \mathcal{O}(f(n))$. n is problem size

Norms: $\|\mathbf{x}\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}, p \ge 1$ $\|\mathbf{x}\|_\infty := \max_{1\le i\le n} |x_i|$ $\|\mathbf{x}\|_2 := (\sum_{i=1}^n |x_i|^2)^{1/2}$ $\|\mathbf{x}\|_1 := \sum_{i=1}^n |x_i|$

1. $\|\mathbf{x}\|_p \ge 0$ $\|\mathbf{x}\|_p = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$
2. $\|\alpha\mathbf{x}\|_p = |\alpha|\|\mathbf{x}\|_p$ for $\alpha \in \mathbb{R}$
3. $\|\mathbf{x}+\mathbf{y}\|_p \le \|\mathbf{x}\|_p + \|\mathbf{y}\|_p$ (Triangle inequality)
4. **Induced Norm**: $\|\mathbf{A}\|_p := \max_{\mathbf{x}\neq 0} \frac{\|\mathbf{Ax}\|_p}{\|\mathbf{x}\|_p}$

1. $\|\mathbf{A}_{m\times n}\|_\infty := \max_{1\le j\le m} \sum_{i=1}^n |a_{ij}|$ (maximum row sum)
2. $\|\mathbf{A}_{m\times n}\|_1 := \max_{1\le i\le n} \sum_{j=1}^m |a_{ij}|$ (maximum column sum)
3. $\|\mathbf{A}_{m\times n}\|_2 := \sqrt{\rho(\mathbf{A}^\top\mathbf{A})}$ $\rho(\mathbf{A}^\top\mathbf{A}) = \max\{|\lambda| : \lambda \text{ is eigenvalue of } \mathbf{A}^\top\mathbf{A}\}$

2 Methods for Systems of Linear Equations (SLE)

Ideal: We want to solve SLE: $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}_{n\times n}$ is invertible, $\mathbf{x}, \mathbf{b} \in \mathbb{R}^n$

2.1 LU Factorization | Gaussian Elimination

Def of LU: If $\mathbf{A}_{n\times n}$ is invertible, then $^1\exists\, \mathbf{L}$: 单位下三角, $^2\exists\, \mathbf{U}$: 可逆上三角. s.t. $\mathbf{A} = \mathbf{LU}$ **(unique)**

LU Factorization: By Gaussian elimination, $\mathbf{A}_{n\times n} = (\prod_{k=1}^{n-1} \mathbf{L}_k)^{-1}\mathbf{U} =: \mathbf{LU}$ $\mathbf{L}_k$: Lower triangular with 1s on the diagonal (by row operation on single column)

1. 如果 $\mathbf{L}$ 是单位下三角矩阵, 且只有第 k 列有非零元素在对角线下, 则 $\mathbf{L}^{-1}$ 矩阵是 $\mathbf{L}$ 对角线不动, 其余为相反数.
2. 如果 $\mathbf{L}_1, \mathbf{L}_2$ 矩阵是单位下三角矩阵, 且 $\mathbf{L}_1|\mathbf{L}_2$ 在第 $1 \sim k|k+1 \sim n$ 列有非零元素在对角线下, 则 $\mathbf{L}_1\mathbf{L}_2$ 矩阵是他们的" 拼接".
3. By LU factorization, $\mathbf{A} = \mathbf{LU}$ $\Rightarrow$ By solving $\mathbf{Ly} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{y}$, we can solve SLE $\mathbf{Ax} = \mathbf{b}$. 用 **FS** 和 **BS** 算法

$$e.g.\ (1)\quad \begin{bmatrix}1&0&0\\2&1&0\\3&0&1\end{bmatrix}^{-1} = \begin{bmatrix}1&0&0\\-2&1&0\\-3&0&1\end{bmatrix} \qquad (2)\quad \begin{bmatrix}1&0&0\\2&1&0\\4&0&1\end{bmatrix}\begin{bmatrix}1&0&0\\0&1&0\\0&5&1\end{bmatrix} = \begin{bmatrix}1&0&0\\2&1&0\\4&5&1\end{bmatrix}$$

Error Analysis of GE: Gaussian elimination is an **unstable** algorithm. (i.e. floating point arithmetic can have a disastrous effect on the computed solution.)

3 Algorithms

DP Dot product $C = \sum_{i=1}^n 2 = 2n = \mathcal{O}(n)$

Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$
Output: $s = \mathbf{x}\cdot\mathbf{y} = \mathbf{x}^\top\mathbf{y}$

$s = 0$
for $i = 1, \dots, n$ **do**
  $s = s + x_i y_i$
end for

MV Matrix-vector multiplication $C = \sum_{i=1}^m 2n = 2nm = \mathcal{O}(mn)$

Input: $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{x} \in \mathbb{R}^n$
Output: $\mathbf{y} = \mathbf{Ax}$

for $i = 1, \dots, m$ **do**
  $y_i = \mathbf{a}_i^\top\mathbf{x}$ # **DP** Algorithm
end for

MM Matrix-matrix multiplication $C = \sum_{i=1}^m \sum_{j=1}^p 2n = 2mnp = \mathcal{O}(mnp)$

Input: $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{B} \in \mathbb{R}^{n\times p}$
Output: $\mathbf{C} = \mathbf{AB}$

for $i = 1, \dots, m$ **do**
  for $j = 1, \dots, p$ **do**
    $c_{ij} = \mathbf{a}_i^\top\mathbf{b}_j$ # **DP** Algorithm
  end for
end for

LU LU factorisation $C(n) = \sum_{k=1}^{n-1}(n-k)(1+2(n-k+1)) = \frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$

Input: $\mathbf{A} \in \mathbb{R}^{n\times n}$
Output: $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{n\times n}$ s.t. $\mathbf{A} = \mathbf{LU}$

$\mathbf{L} = \mathbf{I}$, $\mathbf{U} = \mathbf{A}$
for $k = 1, \dots, n-1$ **do**
  for $j = k+1, \dots, n$ **do**
    $l_{jk} = u_{jk}/u_{kk}$
    $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$
  end for
end for

FS Forward substitution $C(n) = \sum_{j=1}^n [2(j-1)+1] = n^2 = \mathcal{O}(n^2)$

Input: $\mathbf{L} \in \mathbb{R}^{n\times n}$ (lower triangular), $\mathbf{b} \in \mathbb{R}^n$
Output: $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{Ly} = \mathbf{b}$

for $j = 1, \dots, n$ **do**
  $y_j = \frac{1}{l_{jj}}\left(b_j - \sum_{k=1}^{j-1} l_{jk}y_k\right)$
end for

BS Back substitution $C(n) = \sum_{j=1}^n [2(n-j)+1] = n^2 = \mathcal{O}(n^2)$

Input: $\mathbf{U} \in \mathbb{R}^{n\times n}$ (upper triangular), $\mathbf{y} \in \mathbb{R}^n$
Output: $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{Ux} = \mathbf{y}$

for $j = n, \dots, 1$ **do**
  $x_j = \frac{1}{u_{jj}}\left(y_j - \sum_{k=j+1}^n u_{jk}x_k\right)$
end for

GE Gaussian elimination $C(n) = \frac{2}{3}n^3 + \frac{5}{2}n^2 - \frac{7}{6}n = \mathcal{O}(n^3)$

Input: $\mathbf{A} \in \mathbb{R}^{n\times n}$, $\mathbf{b} \in \mathbb{R}^n$
Output: $\mathbf{x} \in \mathbb{R}^n$ s.t. $\mathbf{Ax} = \mathbf{b}$

Find LU factorisation of $\mathbf{A}$. # **LU** Algorithm
Solve $\mathbf{Ly} = \mathbf{b}$ using forward substitution. # **FS** Algorithm
Solve $\mathbf{Ux} = \mathbf{y}$ using back substitution. # **BS** Algorithm

4 Appendix

4.1 Useful Theorems/Formulas

Sum Notation: $\sum_{k=1}^n k = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$ $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n$ $\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2$

4.2 Others

Where the error come from calucation:

1. If your calculation involves 15 or more significant digits, you will likely encounter rounding errors. (Since $\varepsilon_m \approx 10^{-16}$ (relative accuracy), which is about 15 significant digits.)
2. **For GE Algorithm**: It is an unstable algorithm. e.g. $\mathbf{A} = [10^{-20}\ 1\ \backslash\backslash\ 1\ 1] \Rightarrow$ By **GE** Algorithm, $\tilde{\mathbf{L}}\tilde{\mathbf{U}} = \mathbf{A} + [0\ 0\ \backslash\backslash\ 0\ -1]$ (error)